THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG CAMERA AI BÁO CHÁY THỜI GIAN THỰC TRÊN FPGA BASYS 3

Chuyên ngành: Kỹ thuật Máy tính / Điện tử - Viễn thông / Hệ thống Nhúng | Nền tảng: Xilinx Basys 3 (Artix-7 XC7A35T)

I. TỔNG QUAN VỀ DỰ ÁN

A. Đặt vấn đề

Trong những năm gần đây, nguy cơ cháy nổ tại các khu công nghiệp, tòa nhà và các khu vực có mật độ hoạt động cao ngày càng đặt ra yêu cầu về các giải pháp phát hiện cháy nhanh chóng và hiệu quả. Các phương pháp phát hiện cháy truyền thống dựa trên cảm biến khói hoặc nhiệt vẫn còn tồn tại một số hạn chế, đặc biệt là thời gian phản ứng và phạm vi bao quát. Trong khi đó, các giải pháp xử lý hình ảnh kết hợp trí tuệ nhân tạo (AI) có khả năng phân tích trực tiếp dữ liệu video để nhận diện sớm các dấu hiệu của ngọn lửa và làn khói, qua đó mở ra hướng tiếp cận phù hợp cho các hệ thống phát hiện cháy tự động.

Cùng với sự phát triển của xu hướng tính toán tại biên (Edge AI), việc triển khai các mô hình AI trực tiếp trên nền tảng FPGA mang lại khả năng giảm độ trễ, tối ưu mức tiêu thụ năng lượng và tăng khả năng xử lý theo thời gian thực so với các hệ thống dựa trên CPU/GPU trong một số ứng dụng chuyên biệt. Đồng thời, sự phát triển của các thiết bị camera giám sát và nền tảng phần cứng FPGA tạo điều kiện thuận lợi để xây dựng các hệ thống phát hiện cháy thông minh, có khả năng xử lý dữ liệu trực tiếp tại thiết bị và hoạt động liên tục với độ tin cậy cao.

Xuất phát từ những yêu cầu trên, đề tài tập trung nghiên cứu và thiết kế một hệ thống phát hiện cháy sử dụng camera kết hợp với mạng nơ-ron tích chập (CNN) được triển khai trên FPGA. Hệ thống hướng tới khả năng xử lý và nhận diện hình ảnh trực tiếp tại thiết bị, từ đó nâng cao khả năng phát hiện sớm các dấu hiệu cháy và hỗ trợ đưa ra cảnh báo theo thời gian thực.

B. Mục tiêu đề tài

- Xây dựng mạng nơ-ron nhân tạo (CNN) rút gọn phù hợp với tài nguyên phần cứng giới hạn của bo mạch Basys 3.
- Thiết kế các bộ IP Core RTL điều khiển Camera (OV7670), bộ đệm khung hình (Frame Buffer), bộ tăng tốc AI (AI Accelerator) và bộ xuất hình VGA.
- Đạt tốc độ xử lý khung hình thời gian thực (≥ 15-30 FPS) và đưa ra cảnh báo tức thì qua còi (Buzzer) / LED khi có sự cố.

- Xây dựng khối tiền xử lý ảnh (Preprocessing Unit) thực hiện cắt vùng quan tâm (ROI), thay đổi kích thước ảnh và chuẩn hóa dữ liệu nhằm tối ưu đầu vào cho mạng CNN.
- Triển khai, tích hợp và kiểm thử toàn bộ hệ thống trên FPGA Basys 3, đảm bảo các khối chức năng hoạt động đồng bộ và ổn định trong thời gian thực.
- Đánh giá hiệu năng của hệ thống thông qua các tiêu chí như độ chính xác nhận diện, tốc độ xử lý khung hình (FPS), mức sử dụng tài nguyên FPGA và khả năng đáp ứng cảnh báo

C. Phạm vi & Giới hạn đề tài

Phạm vi: Đề tài tập trung vào việc nhận diện hình ảnh ngọn lửa và khói trong môi trường thực tế hoặc thông qua các video mô phỏng. Hệ thống thực hiện phát hiện cháy dựa trên dữ liệu hình ảnh đơn (frame-based), trong đó quá trình suy luận được thực hiện trực tiếp trên FPGA. Kết quả nhận diện được sử dụng để đưa ra cảnh báo tại chỗ thông qua LED/Buzzer và có thể được hiển thị thông qua màn hình VGA theo kiến trúc hệ thống được triển khai.

Giới hạn: Hệ thống sử dụng các tài nguyên nội tại của FPGA Artix-7 XC7A35T và không sử dụng RAM ngoài DDR. Do giới hạn về tài nguyên phần cứng, kích thước đầu vào của mô hình được tối ưu hóa, với các kích thước có thể sử dụng như 64 × 64 hoặc 128 × 128 pixel. Đề tài chỉ triển khai mô hình CNN đã được huấn luyện trước theo quy trình offline, không thực hiện quá trình huấn luyện hoặc cập nhật trọng số trực tiếp trên FPGA.

II. CƠ SỞ LÝ THUYẾT & CÔNG NGHỆ

A. Tổng quan về FPGA Basys 3 & Tài nguyên phần cứng

Bo mạch Digilent Basys 3 sử dụng FPGA Xilinx Artix-7 XC7A35T-1CPG236C, cung cấp nền tảng phần cứng cho việc triển khai hệ thống xử lý ảnh và nhận diện cháy bằng mạng CNN. FPGA cung cấp các tài nguyên phần cứng gồm LUT, Flip-Flop, Block RAM (BRAM) và DSP Slice, cho phép hiện thực các khối xử lý ảnh, bộ nhớ trung gian và các phép toán số học của mạng CNN trực tiếp trên phần cứng.

Trong hệ thống, LUT và Flip-Flop được sử dụng để xây dựng các mạch logic điều khiển, bộ sinh địa chỉ và các thành phần xử lý dữ liệu. BRAM được sử dụng để lưu trữ ảnh sau tiền xử lý và các dữ liệu trung gian cần thiết cho quá trình suy luận CNN. Các DSP48E1 có thể được sử dụng để tăng tốc các phép toán nhân-tích lũy trong các lớp Convolution và Fully Connected.

Basys 3 đóng vai trò là bộ xử lý Edge AI trung tâm. Toàn bộ quá trình từ thu nhận ảnh, tiền xử lý, lưu trữ ảnh, suy luận CNN đến tạo kết quả nhận diện đều được thực hiện trên FPGA. Việc xử lý trực tiếp trên FPGA giúp giảm lượng dữ liệu cần truyền ra bên ngoài và giảm độ trễ trong quá trình phát hiện cháy.

Tham khảo tại **Digilent** *Basys 3 AMD Artix-7 FPGA Trainer Board*, **AMD**

B. Giao tiếp Camera & dữ liệu hình ảnh

Camera OV7670 được sử dụng làm thiết bị thu nhận hình ảnh đầu vào cho hệ thống. Camera sử dụng giao tiếp SCCB để FPGA cấu hình các thanh ghi bên trong cảm biến. Thông qua SCCB, FPGA thiết lập các thông số cần thiết như độ phân giải, định dạng dữ liệu và chế độ hoạt động của camera.

Sau khi được cấu hình, OV7670 xuất dữ liệu hình ảnh sang FPGA thông qua giao tiếp dữ liệu song song. Các tín hiệu DATA[7:0], PCLK, HREF và VSYNC được sử dụng để truyền và đồng bộ dữ liệu hình ảnh.

Trong cấu hình RGB565, hai byte dữ liệu liên tiếp được ghép thành một pixel 16-bit, gồm 5 bit Red, 6 bit Green và 5 bit Blue. Dữ liệu RGB565 sau đó được FPGA tiếp nhận và thực hiện các bước tiền xử lý trước khi đưa vào mạng CNN.

Tham khảo tại **OmniVision**

C. Mạng Nơ-ron Nhân tạo (CNN) và Kỹ thuật Lượng hóa

Mạng CNN được sử dụng để phân loại hình ảnh thành hai trạng thái **Fire** và **No Fire**. Kiến trúc CNN được triển khai trên FPGA gồm các lớp xử lý chính như **Convolution, Max Pooling và Fully Connected**.

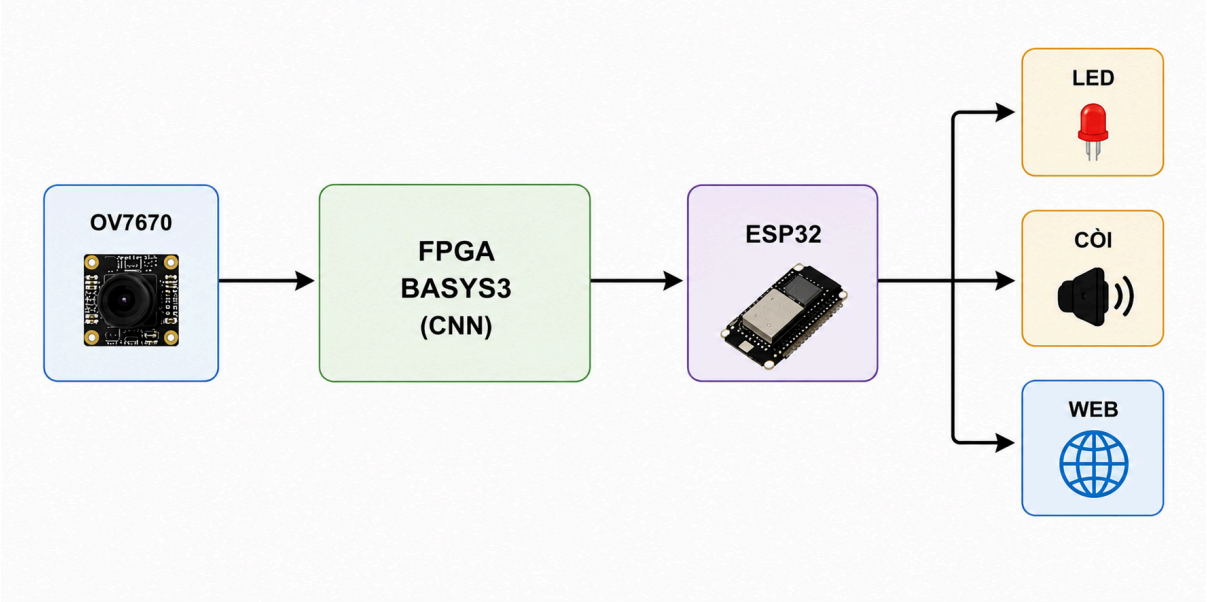
Để phù hợp với tài nguyên phần cứng của FPGA, mô hình CNN được lượng tử hóa trước khi triển khai. Các trọng số và dữ liệu kích hoạt được chuyển từ dạng số thực sang dạng số nguyên có độ rộng bit phù hợp, chẳng hạn Int8 hoặc Fixed-point.

Lượng tử hóa giúp giảm dung lượng bộ nhớ cần thiết, giảm độ phức tạp của phép tính và tạo điều kiện ánh xạ các phép toán CNN lên tài nguyên phần cứng của Artix-7.

Tham khảo tại **IEEE Xplore**, **arXiv**

III. THIẾT KẾ HỆ THỐNG (SYSTEM ARCHITECTURE)

A. Sơ đồ khối tổng thể hệ thống (System Block Diagram)



Toàn bộ hệ thống được tổ chức theo kiến trúc Edge AI, trong đó FPGA Basys 3 thực hiện quá trình xử lý ảnh và suy luận CNN, còn ESP32 đảm nhiệm việc tiếp nhận kết quả và thực

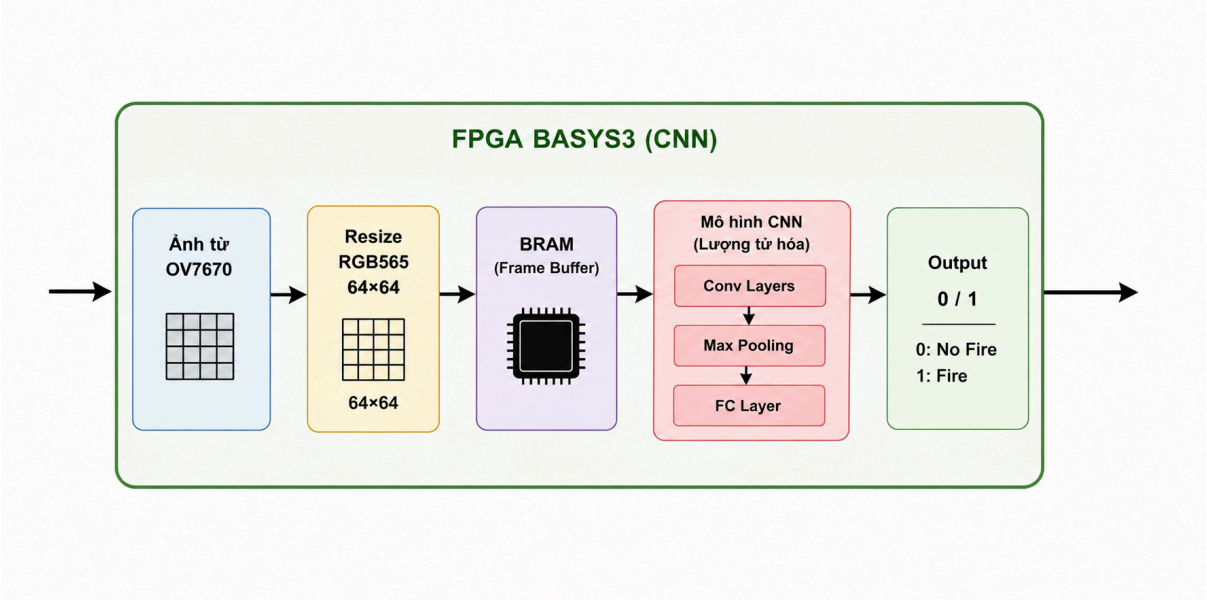
hiện các chức năng cảnh báo, giám sát.

Luồng hoạt động tổng thể:

OV7670 → FPGA Basys 3 → Output 0/1 → ESP32 → Web, LED + Buzzer

Như vậy, FPGA là khối xử lý AI chính, thực hiện toàn bộ quá trình xử lý hình ảnh và CNN. ESP32 chỉ nhận kết quả 0/1 từ FPGA và đảm nhiệm chức năng cảnh báo, truyền thông và giám sát Web.

B. Sơ đồ nguyên lý tổng thể hệ thống (System Schematic)



Toàn bộ hệ thống có thể chia thành hai phần chính.

Phần xử lý AI trên FPGA

OV7670 → Camera Interface → Resize 64×64 RGB565 → BRAM → CNN → Output 0/1

Trong đó:

- Camera Interface: nhận dữ liệu hình ảnh từ OV7670.
- Resize: chuyển ảnh về kích thước đầu vào cố định 64 × 64 pixel.
- BRAM: lưu trữ ảnh RGB565 sau Resize.
- CNN: thực hiện quá trình suy luận.
- Output: tạo kết quả phân loại Fire/No Fire dưới dạng 1 bit.

Phần điều khiển và giám sát

FPGA Output 0/1 → ESP32 → Web

và:

FPGA Output 0/1 → ESP32 → LED + Buzzer
Khi FPGA xác định: Output = 0 → No Fire
ESP32 duy trì hệ thống ở trạng thái bình thường.
Khi: Output = 1 → Fire
ESP32 kích hoạt LED và Buzzer, đồng thời gửi trạng thái cảnh báo lên Web.

C. Thiết kế chi tiết các khối chức năng

Khối Thu Nhận Hình Ảnh (Image Acquisition Unit)



Khối Thu Nhận Hình Ảnh là khối đầu vào của hệ thống, có nhiệm vụ **thu nhận hình ảnh từ camera OV7670 và đưa dữ liệu ảnh vào FPGA Basys 3** để thực hiện các bước xử lý tiếp theo. Camera OV7670 là cảm biến CMOS có giao tiếp dữ liệu song song, phù hợp với yêu cầu thu nhận hình ảnh trực tiếp trong các hệ thống xử lý ảnh trên FPGA. Đối tượng được quan tâm trong hệ thống là **ngọn lửa**, phục vụ cho bài toán phân loại **cháy/không cháy**.

Camera OV7670 truyền dữ liệu hình ảnh tới FPGA thông qua giao tiếp **DVP (Digital Video Port)**. Đường dữ liệu hình ảnh sử dụng bus **DATA[7:0]**, kết hợp với các tín hiệu đồng bộ **PCLK, HREF và VSYNC**. Trong đó, **PCLK (Pixel Clock)** cung cấp xung đồng bộ để FPGA lấy mẫu dữ liệu từ camera, **HREF (Horizontal Reference)** xác định khoảng thời gian dữ liệu của một dòng ảnh hợp lệ, còn **VSYNC (Vertical Synchronization)** xác định ranh giới giữa các frame hình ảnh. FPGA dựa trên các tín hiệu này để xác định thời điểm dữ liệu trên bus DATA[7:0] có giá trị hợp lệ và thực hiện quá trình thu nhận.

Bên cạnh đường truyền dữ liệu DVP, camera OV7670 được **cấu hình thông qua giao tiếp SCCB (Serial Camera Control Bus)**. SCCB được FPGA sử dụng để ghi các giá trị vào các thanh ghi cấu hình bên trong OV7670. Thông qua SCCB, FPGA có thể thiết lập các thông số cần thiết cho camera như **định dạng dữ liệu RGB565, chế độ hoạt động và các tham số độ phân giải phù hợp với hệ thống**. SCCB chỉ được sử dụng cho **quá trình cấu hình và điều khiển camera**, trong khi dữ liệu hình ảnh được truyền qua bus DVP.

Trong quá trình hoạt động, OV7670 xuất dữ liệu pixel theo từng byte trên **DATA[7:0]**. Với định dạng **RGB565**, hai byte liên tiếp được FPGA ghép lại thành một pixel có độ rộng **16 bit**, gồm **5 bit Red, 6 bit Green và 5 bit Blue**. Việc sử dụng RGB565 giúp giảm lượng dữ liệu cần lưu trữ và xử lý so với các định dạng màu có độ sâu lớn hơn, đồng thời phù hợp với tài nguyên phần cứng của FPGA Artix-7.

Camera Interface bên trong FPGA thực hiện việc **đồng bộ dữ liệu theo PCLK, kiểm tra HREF và VSYNC, nhận DATA[7:0] và ghép hai byte thành pixel RGB565**. Các pixel hợp lệ sau khi được tạo sẽ được cung cấp cho quá trình lưu trữ ảnh bên trong FPGA. Quá trình thu nhận được thực hiện liên tục theo từng dòng và từng frame, bảo đảm dữ liệu ảnh được duy trì đúng thứ tự trong quá trình xử lý.

Như vậy, **Khối Thu Nhận Hình Ảnh chỉ đảm nhiệm việc cấu hình và thu nhận dữ liệu từ OV7670**, tạo ra dữ liệu pixel RGB565 hợp lệ cho các khối xử lý bên trong FPGA. Khối sử dụng **SCCB để cấu hình camera** và **DVP để truyền dữ liệu hình ảnh**, không sử dụng UART trong giao tiếp giữa camera và FPGA.

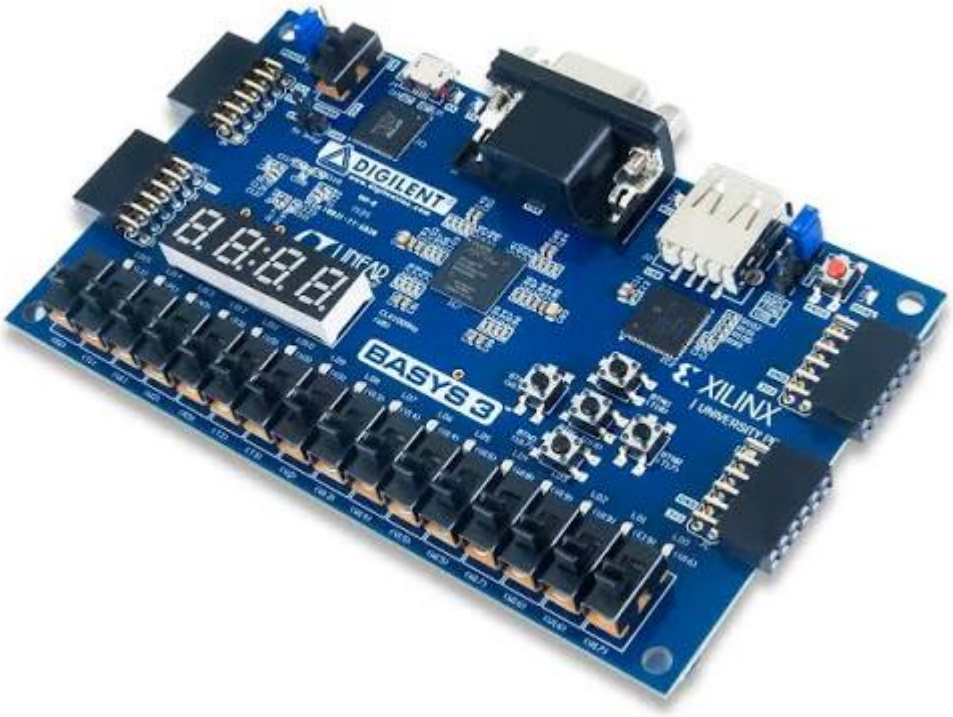
Đầu vào: Hình ảnh từ môi trường và các tín hiệu DVP **DATA[7:0], PCLK, HREF, VSYNC** từ OV7670.

Giao tiếp cấu hình: SCCB giữa FPGA và các thanh ghi cấu hình của OV7670.

Xử lý: Nhận dữ liệu DVP → đồng bộ theo PCLK/HREF/VSYNC → ghép hai byte → tạo pixel **RGB565 16-bit**.

Đầu ra: Dữ liệu pixel **RGB565** hợp lệ đưa vào hệ thống xử lý ảnh trên FPGA.

Khối Xử Lý Trung Tâm FPGA (FPGA Processing Unit)



FPGA Processing Unit là khối xử lý trung tâm của toàn bộ hệ thống, được triển khai trên bo mạch **Digilent Basys 3**, sử dụng FPGA **Xilinx Artix-7 XC7A35T**. Khối này đảm nhiệm toàn bộ quá trình xử lý hình ảnh và suy luận CNN, từ khi tiếp nhận dữ liệu hình ảnh từ camera **OV7670** cho đến khi tạo ra kết quả phân loại **0/1**, tương ứng với hai trạng thái **không cháy/cháy**. FPGA đóng vai trò là nền tảng **Edge AI**, thực hiện quá trình nhận diện trực tiếp tại thiết bị thay vì truyền hình ảnh lên máy chủ để xử lý.

Dữ liệu hình ảnh được truyền từ **OV7670 vào FPGA thông qua giao tiếp DVP**, sử dụng bus dữ liệu **DATA[7:0]** cùng các tín hiệu **PCLK, HREF và VSYNC**. Bên cạnh đó, FPGA sử dụng **SCCB** để cấu hình các thanh ghi của OV7670 trước khi camera bắt đầu hoạt động. SCCB phục vụ cho quá trình cấu hình camera, trong khi DVP đảm nhiệm việc truyền dữ liệu hình ảnh từ camera vào FPGA.

Kiến trúc xử lý bên trong FPGA được tổ chức theo chuỗi:

OV7670 → Camera Interface → Resize → RGB565 64×64 → BRAM → CNN → Output 0/1

Trong đó, **Camera Interface** tiếp nhận dữ liệu từ OV7670 và tạo ra các pixel hình ảnh. Dữ liệu ảnh sau đó được xử lý để đưa về kích thước **64 × 64 pixel** và biểu diễn theo định dạng **RGB565**. Hình ảnh 64 × 64 sau khi được chuẩn bị sẽ được lưu trữ trong **BRAM** nội tại của FPGA để cung cấp dữ liệu ổn định cho quá trình suy luận CNN.

BRAM đóng vai trò là bộ nhớ lưu trữ dữ liệu ảnh đầu vào cho mạng CNN. Việc sử dụng BRAM nội tại giúp dữ liệu hình ảnh được lưu trữ và truy cập trực tiếp bên trong FPGA, không cần sử dụng bộ nhớ ngoài. Điều này phù hợp với kiến trúc Edge AI và giúp giảm độ

trễ trong quá trình truyền dữ liệu giữa các khối xử lý.

Sau khi hình ảnh **RGB565 64 × 64** được lưu vào BRAM, dữ liệu được đưa vào **mô hình CNN** được triển khai trực tiếp trên FPGA. Mô hình CNN của hệ thống gồm các nhóm lớp chính **Convolution, Max Pooling và Fully Connected (FC)**. Các lớp này thực hiện quá trình trích xuất đặc trưng và phân loại hình ảnh để xác định hình ảnh đầu vào có thuộc trạng thái cháy hay không.

Trong lớp **Convolution**, các phép nhân và cộng được thực hiện giữa dữ liệu đầu vào và các trọng số của Kernel nhằm trích xuất các đặc trưng của hình ảnh. Các phép tính số học của CNN được triển khai bằng tài nguyên phần cứng của FPGA, trong đó các **DSP48E1** có thể được sử dụng để thực hiện các phép toán nhân-tích lũy **MAC (Multiply-Accumulate)**. Các trọng số của mô hình được lưu trữ trong bộ nhớ nội bộ phù hợp với kiến trúc thiết kế.

Sau các lớp Convolution, **Max Pooling** được sử dụng để giảm kích thước Feature Map và giữ lại các đặc trưng có giá trị lớn. Việc giảm kích thước dữ liệu giúp giảm số lượng phép tính cần thực hiện ở các lớp CNN tiếp theo, đồng thời giảm yêu cầu về bộ nhớ và tài nguyên phần cứng.

Ở giai đoạn cuối của mạng, **Fully Connected (FC) Layer** thực hiện các phép tính trên Feature Vector đã được trích xuất từ các lớp trước đó. Kết quả của lớp FC được sử dụng để đưa ra quyết định phân loại cuối cùng của mô hình.

Để phù hợp với tài nguyên tính toán và bộ nhớ của FPGA Artix-7, mô hình CNN sử dụng **kỹ thuật lượng hóa (Quantization)**. Các trọng số và dữ liệu tính toán được chuyển từ dạng số thực sang dạng số nguyên phù hợp với kiến trúc phần cứng, giúp giảm số lượng bit cần lưu trữ và giảm độ phức tạp của các phép toán. Nhờ đó, mô hình CNN có thể được triển khai trên FPGA với yêu cầu tài nguyên thấp hơn so với việc sử dụng trực tiếp dữ liệu Float32.

Sau khi hoàn thành các lớp **Convolution → Max Pooling → Fully Connected**, CNN tạo ra kết quả phân loại dưới dạng **0 hoặc 1**. Hai giá trị này tương ứng với hai trạng thái của hệ thống:

- **0: Không cháy (No Fire)**
- **1: Cháy (Fire)**

Kết quả **0/1** là đầu ra nhận diện chính của FPGA. Khi kết quả bằng **1**, FPGA xác định hệ thống đang ở trạng thái cảnh báo cháy và tạo tín hiệu điều khiển tương ứng cho **LED và Buzzer**. LED được sử dụng để tạo cảnh báo trực quan, trong khi Buzzer tạo cảnh báo bằng âm thanh tại vị trí lắp đặt. Khi kết quả bằng **0**, hệ thống duy trì trạng thái không cảnh báo và tiếp tục xử lý các ảnh tiếp theo.

Đồng thời, kết quả phân loại **0/1** được đưa tới **ESP32** để thực hiện chức năng truyền thông. ESP32 tiếp nhận trạng thái nhận diện từ FPGA và sử dụng kết quả này để phục vụ việc gửi thông tin cảnh báo tới **Web Monitoring System**. Như vậy, FPGA chỉ tập trung vào quá trình **xử lý ảnh và CNN**, trong khi ESP32 đảm nhiệm phần giao tiếp với hệ thống Web.

Trong kiến trúc thực tế của hệ thống, **FPGA không thực hiện xuất hình ảnh VGA và không sử dụng màn hình hiển thị**. Dữ liệu hình ảnh từ OV7670 được xử lý hoàn toàn bên trong FPGA, từ quá trình thu nhận, resize, lưu trữ trong BRAM đến suy luận CNN. Kết quả cuối cùng chỉ được chuyển đổi thành trạng thái **0/1**, qua đó giảm đáng kể lượng dữ liệu cần truyền ra bên ngoài.

Như vậy, **FPGA Processing Unit là khối Edge AI cốt lõi của hệ thống**, chịu trách nhiệm trực tiếp xử lý hình ảnh và thực hiện suy luận CNN trên phần cứng Basys 3.

Khối Truyền Thông ESP32 (Communication Unit)



ESP32 là khối trung gian kết nối giữa **FPGA Basys 3** và **Web Monitoring System**, có nhiệm vụ tiếp nhận kết quả nhận diện từ FPGA, xử lý trạng thái và chuyển tiếp thông tin cảnh báo tới hệ thống Web. Việc tách chức năng truyền thông khỏi FPGA cho phép FPGA tập trung tài nguyên cho quá trình xử lý hình ảnh và suy luận CNN, trong khi ESP32 đảm nhiệm vai trò giao tiếp với hệ thống bên ngoài.

ESP32 nhận **kết quả nhận diện 0/1 từ FPGA**, trong đó giá trị **0** biểu thị trạng thái **No Fire** và giá trị **1** biểu thị trạng thái **Fire**. Đây là kết quả cuối cùng sau khi dữ liệu hình ảnh từ camera OV7670 đã được FPGA thực hiện tiền xử lý, lưu trữ và đưa qua mô hình CNN. ESP32 không thực hiện thu nhận hoặc xử lý hình ảnh và không thực hiện suy luận CNN mà chỉ tiếp nhận kết quả nhận diện từ FPGA.

Khi nhận được kết quả **1 (Fire)**, ESP32 xác định hệ thống đang ở trạng thái cảnh báo cháy. Dựa trên trạng thái này, ESP32 thực hiện các chức năng cảnh báo được thiết kế trong hệ thống, đồng thời gửi thông tin cảnh báo tới **Web Monitoring System**. Web có thể sử dụng trạng thái nhận được để hiển thị tình trạng cảnh báo cho người dùng.

Khi nhận được kết quả **0 (No Fire)**, ESP32 xác định hệ thống đang ở trạng thái bình thường và không kích hoạt trạng thái cảnh báo cháy. ESP32 tiếp tục duy trì quá trình giám sát và chờ kết quả nhận diện tiếp theo từ FPGA.

Bên cạnh việc tiếp nhận kết quả nhận diện, ESP32 đóng vai trò là **communication gateway** giữa FPGA và hệ thống Web. ESP32 có thể xử lý và đóng gói trạng thái **Fire/No Fire** thành dữ liệu phù hợp với giao thức truyền thông mà Web sử dụng. Nhờ đó, phần xử lý CNN và phần truyền thông được tách biệt, giúp kiến trúc hệ thống rõ ràng và giảm yêu cầu xử lý giao tiếp trên FPGA.

Trong kiến trúc của hệ thống, **SCCB chỉ được sử dụng cho quá trình cấu hình camera OV7670 với FPGA** và không thuộc chức năng giao tiếp của ESP32. ESP32 không sử dụng module SIM A7680C và hệ thống không sử dụng UART cho đường giao tiếp giữa camera và FPGA. ESP32 chỉ tiếp nhận **kết quả 0/1** từ FPGA và thực hiện chức năng truyền thông tới Web theo giao diện kết nối được thiết kế.

Như vậy, ESP32 tạo thành lớp trung gian giữa **FPGA Edge AI** và **Web Monitoring System**. FPGA chịu trách nhiệm xử lý ảnh và suy luận CNN để tạo kết quả **0/1**, trong khi ESP32 tiếp nhận kết quả, xác định trạng thái **Fire/No Fire**, thực hiện chức năng cảnh báo và truyền thông tin tới Web.

Đầu vào: Kết quả nhận diện **0/1** từ FPGA Basys 3.

Xử lý: Tiếp nhận kết quả → xác định **Fire/No Fire** → điều khiển trạng thái cảnh báo → đóng gói và truyền dữ liệu.

Đầu ra: Tín hiệu điều khiển cảnh báo và dữ liệu trạng thái tới **Web Monitoring System**.

Khối Cảnh Báo Cục Bộ (Local Alarm Unit)

Khối Cảnh Báo Cục Bộ đảm nhiệm chức năng phát tín hiệu cảnh báo trực tiếp tại vị trí lắp đặt khi hệ thống xác định trạng thái **cháy**. Khối sử dụng hai phần tử cảnh báo chính là **LED và Buzzer**, được điều khiển dựa trên kết quả nhận diện từ FPGA thông qua **ESP32**. LED tạo tín hiệu cảnh báo bằng ánh sáng, trong khi Buzzer tạo tín hiệu cảnh báo bằng âm thanh, giúp người vận hành có thể nhận biết sự kiện cháy ngay tại vị trí lắp đặt.

Khối Cảnh Báo Cục Bộ sử dụng **kết quả đầu ra 0/1 từ FPGA** làm cơ sở xác định trạng thái cảnh báo, trong đó:

- **0:** Không cháy (No Fire) → ESP32 duy trì LED và Buzzer ở trạng thái không cảnh báo.
- **1:** Cháy (Fire) → ESP32 kích hoạt LED và Buzzer.

Khi kết quả CNN bằng **0**, FPGA xác định không phát hiện cháy và truyền trạng thái tương ứng tới ESP32. ESP32 duy trì LED và Buzzer ở trạng thái bình thường, đồng thời tiếp tục theo dõi các kết quả nhận diện tiếp theo. Khi kết quả CNN bằng **1**, trạng thái Fire được truyền tới ESP32. ESP32 xác định có cảnh báo cháy và tạo tín hiệu điều khiển để kích hoạt đồng thời **LED và Buzzer**. LED cung cấp cảnh báo trực quan, trong khi Buzzer phát tín hiệu âm thanh nhằm thu hút sự chú ý của người ở gần thiết bị.

Khối Cảnh Báo Cục Bộ **không thực hiện quá trình nhận diện hình ảnh hoặc suy luận CNN**. Việc xử lý hình ảnh và phân loại Fire/No Fire được thực hiện bên trong FPGA. Sau khi mô hình CNN hoàn thành quá trình suy luận và tạo kết quả **0/1**, kết quả được chuyển tới ESP32. ESP32 sử dụng trạng thái này để điều khiển LED và Buzzer. Cách phân chia chức

năng giúp tách biệt rõ ràng giữa **chức năng nhận diện trên FPGA**, **chức năng giao tiếp trên ESP32** và **chức năng cảnh báo tại thiết bị**.

Cảnh báo cục bộ được thực hiện dựa trên trạng thái nhận diện từ FPGA và không phụ thuộc vào việc Web Monitoring System có đang hoạt động hay không. Khi ESP32 nhận được **Output = 1**, LED và Buzzer có thể được kích hoạt tại thiết bị để phát cảnh báo. Do đó, cảnh báo tại chỗ được xử lý ngay sau khi kết quả nhận diện được chuyển từ FPGA sang ESP32, thay vì phụ thuộc vào quá trình hiển thị cảnh báo trên Web.

Khối cảnh báo hoạt động đồng bộ với quá trình nhận diện của hệ thống. Sau khi một ảnh được đưa qua mô hình CNN và tạo ra kết quả **0/1**, trạng thái này được chuyển tới ESP32 để cập nhật trạng thái cảnh báo. ESP32 sau đó tiếp tục xử lý các kết quả nhận diện tiếp theo và điều khiển LED, Buzzer tương ứng. Tùy theo thiết kế điều khiển thực tế, trạng thái cảnh báo có thể được duy trì hoặc cập nhật theo kết quả của các lần suy luận tiếp theo.

Như vậy, Khối Cảnh Báo Cục Bộ đóng vai trò là **lớp phản hồi trực tiếp tại thiết bị**, chuyển kết quả phân loại của CNN thành tín hiệu cảnh báo vật lý thông qua ESP32. Luồng hoạt động của khối được tổ chức:

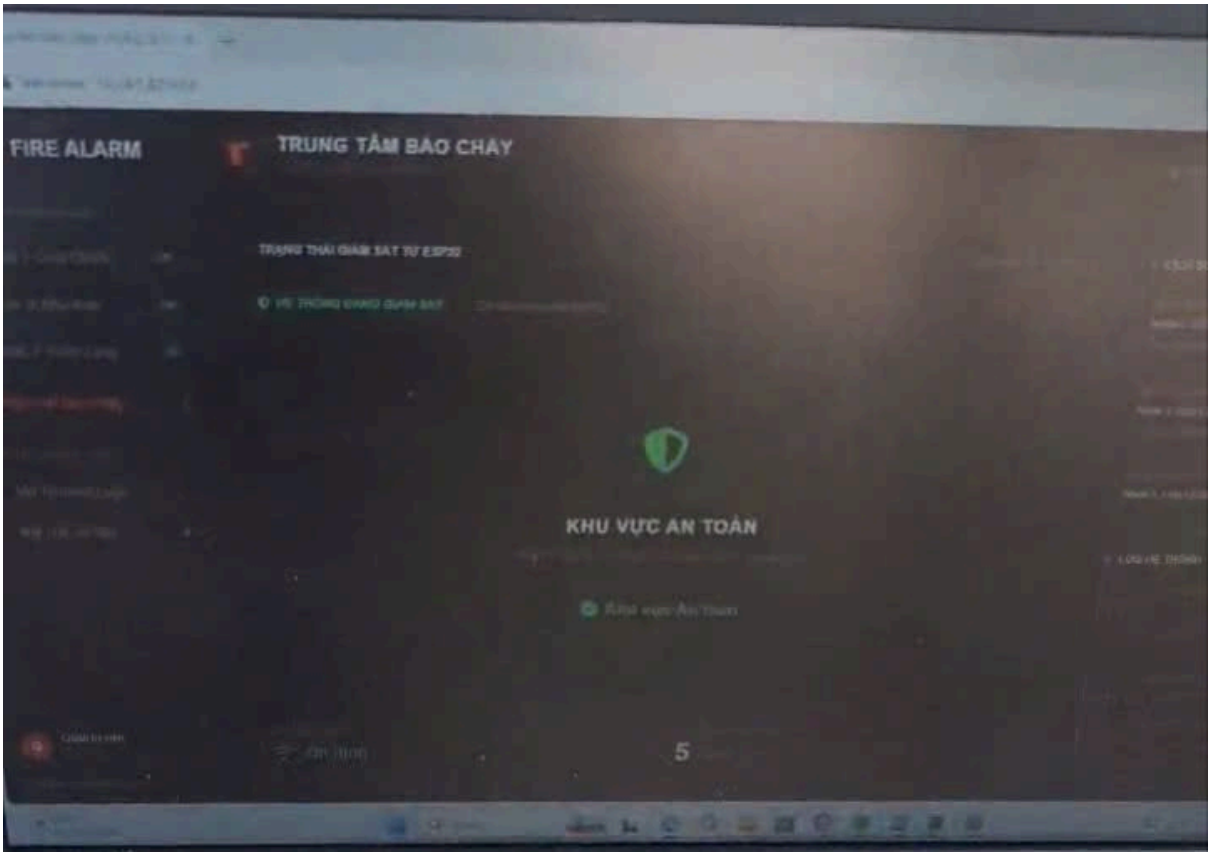
CNN Output 0/1 → ESP32 → LED + Buzzer

Đầu vào: Kết quả phân loại **0/1** từ FPGA.

Xử lý: ESP32 tiếp nhận kết quả → `0` duy trì trạng thái bình thường, `1` kích hoạt cảnh báo.

Đầu ra: Tín hiệu điều khiển **LED và Buzzer**.

Khối Giám Sát Web (Web Monitoring System)



Khối Giám Sát Web (Web Monitoring System) là lớp giao diện phía người dùng, có nhiệm vụ tiếp nhận và hiển thị kết quả nhận diện từ **ESP32**, qua đó cung cấp khả năng theo dõi trạng thái của hệ thống phát hiện cháy từ xa. Khối này không tham gia vào quá trình thu nhận hình ảnh, xử lý ảnh hoặc suy luận CNN mà tập trung vào việc hiển thị trạng thái và các sự kiện cảnh báo do hệ thống tại thiết bị phát hiện.

Sau khi mô hình CNN trên **FPGA Basys 3** hoàn thành quá trình suy luận, kết quả được biểu diễn dưới dạng **0/1**, tương ứng với hai trạng thái **No Fire/Fire**. Kết quả này được đưa tới **ESP32**, sau đó ESP32 xử lý trạng thái và truyền thông tin tới hệ thống Web. Như vậy, Web chỉ tiếp nhận kết quả nhận diện đã được xử lý tại thiết bị thay vì nhận trực tiếp dữ liệu hình ảnh từ camera.

Khi nhận được trạng thái **Fire**, ESP32 gửi thông tin cảnh báo tới Web để cập nhật trạng thái của hệ thống. Giao diện Web có thể hiển thị trạng thái cảnh báo, cho phép người dùng nhận biết hệ thống đang phát hiện cháy. Khi trạng thái là **No Fire**, Web hiển thị trạng thái hoạt động bình thường và tiếp tục chờ các kết quả nhận diện tiếp theo.

Dữ liệu được truyền tới Web có thể bao gồm trạng thái **Fire/No Fire**, thời gian phát hiện và các thông tin sự kiện cần thiết theo thiết kế của hệ thống. Việc chỉ truyền kết quả nhận diện thay vì toàn bộ hình ảnh từ camera giúp giảm lượng dữ liệu cần truyền và phù hợp với kiến trúc **Edge AI**, trong đó quá trình xử lý hình ảnh và suy luận CNN được thực hiện trực tiếp trên FPGA.

Giao diện Web có nhiệm vụ trực quan hóa trạng thái nhận diện để người dùng dễ dàng theo

dõi. Khi hệ thống phát hiện **Fire**, giao diện có thể chuyển sang trạng thái cảnh báo và hiển thị thông tin sự kiện tương ứng. Khi hệ thống ở trạng thái **No Fire**, giao diện duy trì trạng thái bình thường. Các sự kiện nhận diện có thể được hiển thị hoặc lưu lại dưới dạng lịch sử tùy theo thiết kế của phần mềm Web.

Khối Giám Sát Web không thực hiện nhận diện hình ảnh và không quyết định trạng thái cháy. Quyết định **Fire/No Fire** đã được thực hiện bởi mô hình CNN trên FPGA. ESP32 đóng vai trò trung gian tiếp nhận kết quả và truyền dữ liệu tới Web, trong khi Web đảm nhiệm chức năng **hiển thị, giám sát và thông báo trạng thái từ xa**.

Như vậy, luồng dữ liệu của khối được tổ chức theo:

FPGA → ESP32 → Web

Trong đó, FPGA tạo kết quả nhận diện **0/1**, ESP32 tiếp nhận và truyền trạng thái, còn Web hiển thị kết quả cho người dùng.

Đầu vào: Kết quả nhận diện **0/1** được truyền thông qua ESP32.

Xử lý: Tiếp nhận dữ liệu → cập nhật trạng thái → hiển thị thông tin cảnh báo.

Đầu ra: Giao diện giám sát trạng thái **Fire/No Fire** và thông tin cảnh báo trên Web.

D. Thiết kế chi tiết các khối kiến trúc FPGA

Kiến trúc xử lý bên trong FPGA được xây dựng theo mô hình **data processing pipeline**, trong đó dữ liệu hình ảnh được xử lý tuần tự qua các giai đoạn thu nhận, lưu trữ, tiền xử lý, suy luận AI và hậu xử lý. Sau khi có kết quả nhận diện, FPGA đồng thời thực hiện cảnh báo cục bộ thông qua LED/Buzzer và truyền trạng thái nhận diện sang ESP32.

Kiến trúc FPGA gồm **5 khối chức năng chính**:
Camera Interface → Resize → Bram → Mô hình CNN → Output 0/1

Khối Thu Nhận Ảnh (Camera Interface)

Camera Interface là khối giao tiếp trực tiếp giữa camera **OV7670** và FPGA, có nhiệm vụ cấu hình camera và tiếp nhận dữ liệu hình ảnh từ cảm biến. Khối này sử dụng **SCCB (Serial Camera Control Bus)** để thiết lập các thanh ghi điều khiển bên trong OV7670, đồng thời sử dụng giao tiếp dữ liệu song song **DVP (Digital Video Port)** để nhận dữ liệu hình ảnh từ camera. Camera Interface tạo ra dữ liệu pixel **RGB565** và các tín hiệu điều khiển cần thiết để đưa ảnh vào các khối xử lý tiếp theo trên FPGA.

Trong quá trình cấu hình, FPGA sử dụng giao tiếp **SCCB** để ghi các giá trị vào các thanh ghi của OV7670. Thông qua SCCB, các tham số hoạt động của camera như **độ phân giải, định dạng màu và chế độ xuất dữ liệu** được thiết lập theo yêu cầu của hệ thống. Bộ điều khiển SCCB thực hiện tuần tự quá trình gửi địa chỉ thanh ghi và dữ liệu cấu hình. Sau khi quá trình cấu hình hoàn tất, OV7670 chuyển sang trạng thái hoạt động và bắt đầu xuất dữ liệu hình ảnh theo cấu hình đã thiết lập.

Sau khi được cấu hình, OV7670 truyền dữ liệu hình ảnh tới FPGA thông qua giao tiếp **DVP**. Giao diện này sử dụng bus dữ liệu **DATA[7:0]** cùng các tín hiệu **PCLK, HREF và VSYNC**. Trong đó, **PCLK (Pixel Clock)** cung cấp xung đồng bộ để FPGA lấy mẫu dữ liệu trên bus DATA[7:0], **HREF (Horizontal Reference)** xác định khoảng thời gian dữ liệu của một dòng ảnh là hợp lệ, còn **VSYNC (Vertical Synchronization)** xác định ranh giới giữa các frame. Camera Interface sử dụng các tín hiệu này để xác định thời điểm dữ liệu hình ảnh hợp lệ và thực hiện quá trình thu nhận theo đúng cấu trúc của frame.

Trong cấu hình **RGB565**, dữ liệu hình ảnh được truyền trên DATA[7:0] theo từng byte. Camera Interface sử dụng thanh ghi tạm để lưu byte thứ nhất, sau đó kết hợp với byte thứ hai để tạo thành một pixel có độ rộng **16 bit**. Pixel RGB565 được phân bố thành **5 bit Red, 6 bit Green và 5 bit Blue**. Sau khi nhận đủ hai byte hợp lệ, Camera Interface tạo tín hiệu xác nhận pixel hợp lệ và cung cấp dữ liệu RGB565 cho khối xử lý tiếp theo.

Quá trình thu nhận dữ liệu được đồng bộ theo **PCLK**, đồng thời sử dụng **HREF** và **VSYNC** để xác định vị trí của dữ liệu trong frame. Khi HREF ở trạng thái cho phép, các byte dữ liệu trên DATA[7:0] được lấy mẫu theo PCLK. Hai byte liên tiếp được ghép thành một pixel RGB565. VSYNC được sử dụng để xác định thời điểm chuyển sang frame mới, từ đó hỗ trợ việc tổ chức dữ liệu hình ảnh theo đúng thứ tự.

Sau khi tạo được dữ liệu RGB565, Camera Interface cung cấp pixel cùng thông tin vị trí hoặc địa chỉ tương ứng cho quá trình lưu trữ và xử lý ảnh tiếp theo. Dữ liệu này được đưa qua bước **Resize** để chuyển ảnh về kích thước **64 × 64 pixel** theo yêu cầu của mô hình CNN, trước khi được lưu trữ và sử dụng trong các giai đoạn xử lý tiếp theo của FPGA.

Như vậy, Camera Interface thực hiện hai phương thức giao tiếp chính với OV7670: **SCCB dùng để cấu hình camera** và **DVP dùng để truyền dữ liệu hình ảnh**. Đầu vào của khối gồm **DATA[7:0], PCLK, HREF và VSYNC** từ OV7670 cùng các đường tín hiệu SCCB phục vụ cấu hình. Đầu ra của khối là dữ liệu pixel **RGB565**, tín hiệu pixel hợp lệ và thông tin vị trí dữ liệu phục vụ quá trình xử lý ảnh tiếp theo trên FPGA.

Khối Resize

Resize Unit là khối tiền xử lý có nhiệm vụ chuyển đổi kích thước ảnh nhận được từ **OV7670** về kích thước **64 × 64 pixel** trước khi lưu trữ vào **BRAM** và cung cấp cho mô hình CNN. Khối nhận dữ liệu pixel **RGB565** từ Camera Interface và thực hiện ánh xạ các pixel của ảnh đầu vào sang hệ tọa độ của ảnh đầu ra 64 × 64. Việc giảm kích thước ảnh giúp giảm số lượng dữ liệu cần lưu trữ và số phép tính phải thực hiện trong các lớp CNN, đồng thời đưa dữ liệu về đúng kích thước đầu vào được sử dụng trong mô hình.

Dữ liệu đầu vào của Resize Unit là các pixel **RGB565 16 bit** cùng với thông tin vị trí của pixel trong frame. Dựa trên tọa độ hàng và cột của ảnh đầu vào, khối xác định những pixel

cần được sử dụng để tạo ảnh có kích thước 64 × 64. Quá trình này có thể được thực hiện bằng phương pháp **Downsampling**, trong đó nhiều pixel của ảnh đầu vào được ánh xạ vào một pixel tương ứng của ảnh đầu ra. Cách tổ chức này cho phép giảm kích thước ảnh mà không cần sử dụng các phép tính phức tạp.

Resize Unit duy trì các bộ đếm hoặc địa chỉ tương ứng với **tọa độ X và Y** của pixel đầu vào và pixel đầu ra. Khi nhận được một pixel hợp lệ, logic xử lý xác định pixel đó có thuộc vị trí cần lấy mẫu hay không. Nếu pixel được chọn, dữ liệu RGB565 được chuyển tới vị trí tương ứng trong ảnh đầu ra 64 × 64. Địa chỉ của ảnh đầu ra được xác định từ tọa độ hàng và cột theo quan hệ: **Address = Y × 64 + X**

với **X** là tọa độ cột từ 0 đến 63 và **Y** là tọa độ hàng từ 0 đến 63. Nhờ cách đánh địa chỉ này, 4096 pixel của ảnh 64 × 64 được tổ chức tuần tự trong bộ nhớ.

Dữ liệu màu của pixel không bị thay đổi trong quá trình Resize. Pixel sau khi được lựa chọn vẫn giữ nguyên định dạng **RGB565 16 bit**, gồm **5 bit Red, 6 bit Green và 5 bit Blue**. Vì vậy, đầu ra của Resize Unit là ảnh có kích thước **64 × 64 pixel với tổng cộng 4096 pixel**, mỗi pixel có độ rộng 16 bit. Tổng lượng dữ liệu ảnh cần lưu trữ là **4096 × 16 bit**.

Sau khi hoàn thành quá trình ánh xạ, Resize Unit tạo tín hiệu **write enable** và địa chỉ ghi tương ứng để lưu từng pixel RGB565 vào **BRAM**. BRAM đóng vai trò lưu trữ ảnh 64 × 64 sau khi resize, tạo vùng dữ liệu ổn định để các khối xử lý phía sau có thể đọc ảnh theo địa chỉ. Việc tách quá trình resize khỏi quá trình suy luận giúp dữ liệu đầu vào của CNN có kích thước cố định và phù hợp với kiến trúc mô hình.

Quá trình Resize được thực hiện liên tục theo từng frame. Khi bắt đầu một frame mới, các bộ đếm tọa độ và địa chỉ được khởi tạo lại. Trong quá trình nhận dữ liệu, các pixel được xử lý tuần tự theo từng dòng và được ánh xạ vào ảnh đầu ra. Khi đủ **64 × 64 pixel**, khối xác định frame đầu ra đã hoàn thành và dữ liệu có thể được sử dụng cho giai đoạn xử lý tiếp theo.

Như vậy, **Resize Unit** tạo cầu nối giữa dữ liệu hình ảnh RGB565 từ Camera Interface và bộ nhớ ảnh đầu vào của CNN. Đầu vào của khối là các pixel RGB565 từ OV7670 thông qua Camera Interface cùng thông tin vị trí pixel; đầu ra là ảnh **64 × 64 RGB565** được tổ chức theo địa chỉ và ghi vào **BRAM**. Khối này chỉ thực hiện chức năng thay đổi kích thước và tổ chức dữ liệu ảnh, không thực hiện các phép tính của mạng CNN hoặc chức năng cảnh báo và truyền thông.

Khối BRAM (Block RAM)

BRAM (Block RAM) là khối bộ nhớ nội bộ được sử dụng để lưu trữ ảnh **64 × 64 pixel RGB565** sau khi dữ liệu hình ảnh được xử lý bởi Resize Unit. BRAM được triển khai trực

tiếp bằng tài nguyên **Block RAM tích hợp trong FPGA Artix-7**, không yêu cầu sử dụng bộ nhớ RAM bên ngoài. Khối này tạo vùng lưu trữ trung gian giữa quá trình tiền xử lý hình ảnh và quá trình suy luận của mô hình CNN.

Dữ liệu đầu vào của BRAM là các pixel **RGB565 16 bit** từ Resize Unit. Với ảnh có kích thước 64 × 64 pixel, tổng số pixel cần lưu trữ là **4096 pixel**. Mỗi pixel có độ rộng 16 bit nên bộ nhớ cần lưu trữ tối thiểu **4096 × 16 bit**, tương đương **65.536 bit**. Các pixel được ghi tuần tự vào bộ nhớ theo địa chỉ tương ứng với vị trí của chúng trong ảnh.

Địa chỉ của mỗi pixel được xác định dựa trên tọa độ hàng và cột của ảnh. Với ảnh 64 × 64, địa chỉ bộ nhớ có thể được tổ chức theo quan hệ:

Address = Y × 64 + X

trong đó **X** là tọa độ cột và **Y** là tọa độ hàng. Giá trị địa chỉ nằm trong khoảng từ **0 đến 4095**, tương ứng với toàn bộ 4096 pixel của ảnh. Cách tổ chức này giúp dữ liệu ảnh được lưu trữ tuần tự và cho phép khối CNN truy cập từng pixel thông qua địa chỉ tương ứng.

BRAM sử dụng các tín hiệu điều khiển **Write Enable**, **Write Address** và **Write Data** trong quá trình ghi. Khi Resize Unit tạo ra một pixel RGB565 hợp lệ, địa chỉ ghi được cung cấp cho BRAM cùng với dữ liệu pixel và tín hiệu cho phép ghi. BRAM lưu dữ liệu tại địa chỉ tương ứng. Sau khi quá trình ghi hoàn tất, toàn bộ ảnh 64 × 64 được duy trì trong bộ nhớ để phục vụ quá trình suy luận.

Bên cạnh đường ghi, BRAM cung cấp đường đọc dữ liệu cho khối CNN. Bộ điều khiển của CNN tạo **Read Address** để truy cập lần lượt các pixel hoặc các vùng dữ liệu cần thiết cho quá trình tính toán. Dữ liệu đọc từ BRAM được cung cấp cho các lớp xử lý của CNN theo thứ tự phù hợp với kiến trúc mạng. Việc sử dụng bộ nhớ nội bộ giúp dữ liệu hình ảnh được truy cập trực tiếp trên FPGA với độ trễ thấp và giảm nhu cầu truyền dữ liệu ra các thiết bị bên ngoài.

Tùy theo kiến trúc bộ nhớ được sử dụng, BRAM có thể được tổ chức theo dạng **Single-Port hoặc Dual-Port**. Với kiến trúc Dual-Port, quá trình ghi ảnh và đọc dữ liệu có thể sử dụng các cổng bộ nhớ độc lập, cho phép tách biệt hoạt động của Resize Unit và khối CNN. Cách tổ chức này phù hợp với kiến trúc xử lý ảnh trong đó dữ liệu được ghi vào bộ nhớ ở một giai đoạn và được đọc để suy luận ở giai đoạn tiếp theo.

BRAM chỉ lưu trữ dữ liệu ảnh và cung cấp dữ liệu theo địa chỉ yêu cầu. Khối không thực hiện thay đổi kích thước ảnh, không thực hiện các phép tính Convolution, Max Pooling hoặc Fully Connected và không thực hiện quyết định **Fire/No Fire**. Các chức năng này thuộc về các khối xử lý tương ứng trong kiến trúc CNN.

Trong quá trình hoạt động, BRAM được sử dụng cho từng frame ảnh đầu vào của mạng. Khi một frame **64 × 64 RGB565** mới được tạo ra, dữ liệu được ghi vào các địa chỉ tương ứng trong bộ nhớ. Sau khi dữ liệu sẵn sàng, khối CNN có thể đọc các pixel từ BRAM để

thực hiện quá trình suy luận. Việc sử dụng BRAM tạo ra ranh giới rõ ràng giữa giai đoạn lưu trữ ảnh và giai đoạn tính toán CNN, đồng thời tận dụng tài nguyên bộ nhớ tích hợp của FPGA.

Như vậy, BRAM đóng vai trò là **bộ nhớ ảnh đầu vào của CNN**, nhận ảnh **64 × 64 RGB565** từ Resize Unit và cung cấp lại dữ liệu ảnh cho khối CNN thông qua cơ chế địa chỉ đọc. Đầu vào của khối là **RGB565 16 bit, 4096 pixel** cùng địa chỉ và tín hiệu ghi; đầu ra là dữ liệu pixel được đọc theo địa chỉ phục vụ quá trình suy luận CNN.

Khối Mô Hình CNN (CNN Model)

CNN Model là khối thực hiện quá trình suy luận và phân loại ảnh trên FPGA, nhận dữ liệu ảnh **64 × 64 RGB565** từ BRAM và tạo ra kết quả phân loại nhị phân **0/1**, tương ứng với hai trạng thái **No Fire/Fire** của hệ thống. Mô hình được xây dựng từ các lớp tính toán chính gồm **Convolution, Max Pooling và Fully Connected (FC)**. Toàn bộ quá trình suy luận được thực hiện trực tiếp trên FPGA bằng các phép tính số nguyên phù hợp với kiến trúc phần cứng.

Ảnh đầu vào của mô hình là ảnh **64 × 64 pixel với định dạng RGB565 16 bit** đã được lưu trong BRAM. Trong quá trình đưa dữ liệu vào mạng, các giá trị màu của pixel được tách hoặc chuyển đổi theo cách phù hợp với mô hình đã được xây dựng. Dữ liệu sau đó được đưa lần lượt qua các tầng của mạng CNN. Việc sử dụng kích thước đầu vào cố định 64 × 64 giúp xác định trước số lượng dữ liệu cần xử lý và kích thước của các Feature Map trong từng tầng của mạng.

Convolution Layer là tầng đầu tiên thực hiện quá trình trích xuất đặc trưng từ ảnh. Tầng này sử dụng các **Kernel/Weight** đã được huấn luyện để thực hiện phép tích chập trên dữ liệu đầu vào. Với mỗi vị trí của Kernel, các giá trị pixel hoặc Feature Map được nhân với trọng số tương ứng và cộng dồn để tạo thành giá trị của Feature Map đầu ra. Các phép tính này được thực hiện theo dạng **Multiply-Accumulate (MAC)** và được lặp lại trên nhiều vị trí của ảnh để tạo ra các đặc trưng cần thiết cho quá trình phân loại.

Các trọng số của các lớp CNN được lưu trữ dưới dạng dữ liệu số phù hợp với phần cứng FPGA. Trong quá trình suy luận, bộ điều khiển của mô hình lần lượt đọc dữ liệu đầu vào và trọng số tương ứng, sau đó thực hiện các phép tính tích chập. Các Feature Map tạo ra sau mỗi tầng được lưu trữ trong bộ nhớ nội bộ để sử dụng cho tầng tiếp theo. Cách tổ chức này hạn chế việc phải truyền dữ liệu ra ngoài FPGA trong quá trình suy luận.

Sau lớp Convolution, **Max Pooling Layer** thực hiện giảm kích thước Feature Map. Max Pooling chia Feature Map thành các vùng cục bộ và lựa chọn giá trị lớn nhất trong mỗi vùng. Quá trình này làm giảm số lượng phần tử cần xử lý ở các tầng tiếp theo, đồng thời giữ lại các đặc trưng có giá trị lớn đã được trích xuất từ tầng Convolution. Kích thước và bước dịch

của vùng Pooling được xác định theo kiến trúc mô hình CNN được sử dụng.

Các Feature Map sau các tầng Convolution và Max Pooling được đưa vào **Fully Connected Layer**. Trước khi thực hiện phép tính FC, dữ liệu Feature Map được tổ chức thành một **Feature Vector**. Lớp FC thực hiện các phép nhân giữa các phần tử của Feature Vector và các trọng số tương ứng, sau đó cộng dồn các kết quả để tạo ra giá trị đầu ra của mạng. Các phép tính của FC cũng được thực hiện dưới dạng số nguyên sau quá trình lượng tử hóa.

Lượng tử hóa (Quantization) được sử dụng để chuyển các giá trị của mô hình từ dạng biểu diễn có độ chính xác cao sang dạng số nguyên có độ rộng bit phù hợp với tài nguyên FPGA. Quá trình này được áp dụng cho **trọng số và dữ liệu kích hoạt** theo cấu hình của mô hình. Việc sử dụng dữ liệu có độ rộng bit thấp giúp giảm dung lượng bộ nhớ cần thiết và giảm chi phí tính toán so với việc thực hiện trực tiếp bằng số dấu phẩy động.

Các tham số lượng tử hóa được xác định trong quá trình xây dựng mô hình và phải được giữ nhất quán giữa mô hình sử dụng để huấn luyện/đánh giá và phần cứng FPGA. Nhờ đó, các giá trị được sử dụng trong quá trình suy luận phần cứng có cùng cách biểu diễn với mô hình đã được chuẩn bị trước khi triển khai. Trọng số lượng tử hóa có thể được lưu trong **ROM/BRAM**, trong khi các Feature Map trung gian được lưu trữ trong bộ nhớ nội bộ FPGA.

Toàn bộ trình tự tính toán của CNN được điều khiển bởi logic điều khiển của khối. Bộ điều khiển quản lý việc đọc dữ liệu từ BRAM, đọc trọng số, thực hiện các phép tính của từng tầng, lưu kết quả trung gian và chuyển dữ liệu sang tầng tiếp theo. Quá trình được thực hiện tuần tự theo cấu trúc của mô hình, từ **Convolution → Max Pooling → Fully Connected** cho đến khi tạo được kết quả cuối cùng.

Sau khi hoàn thành lớp FC, CNN tạo ra kết quả phân loại của ảnh đầu vào. Kết quả được biểu diễn dưới dạng **1 bit**, trong đó **0 tương ứng với No Fire và 1 tương ứng với Fire**. Bit kết quả này là đầu ra của khối CNN và được đưa tới khối xử lý kết quả tiếp theo để thực hiện các chức năng điều khiển của hệ thống.

Như vậy, khối **CNN Model** nhận ảnh **64 × 64 RGB565** từ BRAM, thực hiện quá trình suy luận thông qua các lớp **Convolution → Max Pooling → Fully Connected**, sử dụng dữ liệu và trọng số đã được **lượng tử hóa**, sau đó tạo ra đầu ra **0/1 (No Fire/Fire)**. Khối chỉ đảm nhiệm chức năng tính toán và phân loại ảnh, không thực hiện giao tiếp SCCB với camera, không điều khiển LED/Buzzer và không đảm nhiệm chức năng truyền thông với ESP32.

Khối Output 0/1 – Fire/No Fire Decision

Output 0/1 là khối kết quả cuối của quá trình suy luận CNN trên FPGA, có nhiệm vụ chuyển kết quả tính toán của mô hình thành một tín hiệu nhị phân biểu diễn trạng thái nhận diện của hệ thống. Đầu vào của khối là kết quả đầu ra từ **Fully Connected Layer** của CNN, sau khi

các phép tính của mô hình đã được hoàn thành. Đầu ra được biểu diễn bằng **1 bit**, tương ứng với hai trạng thái **Fire** và **No Fire**.

Sau khi CNN hoàn thành quá trình xử lý ảnh, kết quả đầu ra được đưa vào logic quyết định của khối Output. Đối với bài toán phân loại nhị phân, giá trị kết quả được sử dụng để xác định lớp dự đoán. Khi kết quả thỏa điều kiện tương ứng với lớp **Fire**, tín hiệu đầu ra được đặt ở mức **1**. Khi kết quả tương ứng với lớp **No Fire**, tín hiệu đầu ra được đặt ở mức **0**.

Tín hiệu đầu ra có thể được biểu diễn như sau:

- **Output = 0:** No Fire – không phát hiện cháy.
- **Output = 1:** Fire – phát hiện cháy.

Việc sử dụng tín hiệu nhị phân giúp đơn giản hóa giao tiếp giữa khối suy luận CNN và các khối điều khiển phía sau. Thay vì truyền toàn bộ Feature Map hoặc các giá trị trung gian của mạng, hệ thống chỉ cần sử dụng một tín hiệu trạng thái để biểu diễn kết quả nhận diện. Tín hiệu này được giữ ổn định cho đến khi có kết quả suy luận mới được xác nhận.

Kết quả Output 0/1 được cập nhật sau khi một lần suy luận hoàn tất nhằm bảo đảm tín hiệu đầu ra phản ánh kết quả của một frame đã được xử lý đầy đủ. Trong quá trình CNN đang tính toán, trạng thái đầu ra hiện tại không được thay đổi bởi các giá trị trung gian. Khi quá trình suy luận hoàn thành, kết quả mới được ghi nhận và trở thành trạng thái nhận diện hiện tại của hệ thống.

Tín hiệu **Fire/No Fire** sau đó được đưa tới khối điều khiển và giao tiếp của hệ thống. Khi **Output = 1**, trạng thái Fire được truyền tới **ESP32** để thực hiện các chức năng phản hồi và thông báo tương ứng. Khi **Output = 0**, ESP32 nhận trạng thái No Fire và hệ thống tiếp tục duy trì quá trình giám sát các frame tiếp theo.

Đối với trạng thái Fire, kết quả từ FPGA được sử dụng để tạo các phản hồi cảnh báo như **bật LED, kích hoạt còi và cập nhật trạng thái cảnh báo trên Web** thông qua ESP32. Các chức năng này không thuộc bản thân khối Output 0/1; khối chỉ cung cấp tín hiệu trạng thái nhận diện làm đầu vào cho các chức năng phía sau.

Khối Output 0/1 không thực hiện quá trình thu nhận hình ảnh, resize, lưu trữ BRAM hoặc tính toán các lớp CNN. Các chức năng này đã được thực hiện ở các khối phía trước. Tương tự, khối cũng không thực hiện giao tiếp SCCB với OV7670 và không trực tiếp điều khiển các chức năng truyền thông hoặc cảnh báo.

Như vậy, **Output 0/1** là điểm chuyển tiếp giữa quá trình **CNN Inference** và hệ thống điều khiển phía sau. Đầu vào của khối là kết quả phân loại từ CNN, đầu ra là tín hiệu nhị phân **0/1**, trong đó **0 biểu thị No Fire và 1 biểu thị Fire**. Tín hiệu này được sử dụng làm cơ sở để ESP32 thực hiện các phản hồi tương ứng, bao gồm **cảnh báo LED, còi và thông báo trên Web**.

IV.KHAI VÀ HIỆN THỰC HÓA (IMPLEMENTATION)

A. Huấn luyện và Lượng hóa Mô hình (Offline Workflow)

- Thu thập và chuẩn bị tập dữ liệu (Dataset) hình ảnh cháy và không cháy.
- Huấn luyện mô hình CNN tối ưu trên Python với PyTorch / TensorFlow.
- Thực hiện Quantization, xuất mảng trọng số dưới dạng file cấu hình bộ nhớ ROM/BRAM (.coe hoặc .hex).
- Tiền xử lý dữ liệu huấn luyện: Gán nhãn (Labeling), thay đổi kích thước ảnh (Resize), chuẩn hóa dữ liệu (Normalization) và tăng cường dữ liệu (Data Augmentation) nhằm nâng cao khả năng tổng quát của mô hình.
- Đánh giá và lựa chọn mô hình dựa trên các chỉ số như Accuracy, Precision, Recall, F1-score và Loss để cân bằng giữa độ chính xác và tài nguyên phần cứng.
- Chuyển đổi mô hình sau lượng hóa thành các tệp dữ liệu phù hợp với FPGA, đồng thời kiểm tra tính nhất quán giữa kết quả suy luận trên phần mềm và dữ liệu trọng số được nạp vào ROM/BRAM.

B. Lập trình Phần cứng RTL trên Xilinx Vivado

- Viết mã mô tả phần cứng bằng ngôn ngữ Verilog HDL / VHDL .
- Tích hợp các IP Core của Vivado: Clocking Wizard (tạo xung 24MHz cho Camera, 25MHz cho VGA), Block Memory Generator.
- Gán chân tín hiệu phần cứng trong file rằng buộc (Basys3_Master.xdc).
- Thiết kế các module RTL theo kiến trúc phân cấp (Hierarchical Design), bao gồm Camera Interface, Frame Buffer, Preprocessing Unit, AI Accelerator, VGA Controller và Alarm Controller.
- Thực hiện mô phỏng chức năng (Functional Simulation), tổng hợp (Synthesis), cài đặt (Implementation) và sinh tệp cấu hình Bitstream để nạp lên FPGA.
- Phân tích báo cáo sử dụng tài nguyên (Resource Utilization) và thời gian (Timing Analysis), tối ưu thiết kế nhằm đáp ứng yêu cầu hoạt động ổn định và xử lý thời gian thực.

V.KẾT QUẢ & ĐÁNH GIÁ

A. Kết quả tổng hợp tài nguyên phần cứng (Resource Utilization)

Tài nguyên FPGA	**Đã sử dụng**	**Khả dụng (Basys 3)**	**Tỷ lệ sử dụng (%)**
LUTs (Logic)	14,500	20,800	69.7%
Flip-Flops (FF)	18,200	41,600	43.7%
Block RAM (BRAM)	45 / 50 RAMB36	50 RAMB36	90.0%
DSP Slices (DSP48E1)	72	90	80.0%

B. Đánh giá hiệu năng hoạt động

- **Độ chính xác AI:** Đạt tỷ lệ nhận diện đúng (Accuracy) ≥ 91% trên tập dữ liệu thử nghiệm.
- **Tốc độ xử lý:** Đạt tốc độ ***24 FPS*** với độ trễ xử lý mỗi khung hình khoảng ***41.6 ms***.
- **Thử nghiệm thực tế:** Thử nghiệm thành công trong các điều kiện ánh sáng khác nhau, nhận diện tốt lửa từ bật lửa, video mô phỏng và nến.

VI. KẾT LUẬN & HƯỚNG PHÁT TRIỂN

A. Kết luận

Đề tài đã hoàn thành việc thiết kế và thực thi toàn bộ luồng xử lý nhận diện **AI báo cháy** trên bo mạch **Digilent Basys 3**, từ quá trình thu nhận dữ liệu hình ảnh, tiền xử lý, suy luận mạng CNN đến hậu xử lý và đưa ra kết quả cảnh báo. Kiến trúc hệ thống được xây dựng theo hướng **Edge AI**, trong đó quá trình suy luận được thực hiện trực tiếp trên FPGA thay vì phụ thuộc vào máy tính hoặc máy chủ bên ngoài.

Kết quả thiết kế cho thấy tính khả thi của việc **nén và triển khai mạng CNN trên FPGA có dung lượng tài nguyên tương đối nhỏ**, thông qua các kỹ thuật lượng hóa dữ liệu và tận dụng các tài nguyên phần cứng như **BRAM, LUT, Flip-Flop và DSP48E1**. Việc tổ chức các khối xử lý theo kiến trúc pipeline và khai thác khả năng xử lý song song của FPGA góp phần đáp ứng yêu cầu xử lý theo thời gian thực, đồng thời giảm độ trễ trong quá trình phát hiện và cảnh báo cháy.

B. Hạn chế & Hướng phát triển

- **Hạn chế:** Do giới hạn BRAM của Artix-7 XC7A35T, mô hình AI phải nén sâu, chưa thể mở rộng lên các độ phân giải cao HD/FullHD.
- **Hướng phát triển:** Nâng cấp lên bo mạch SoC FPGA (như Xilinx Zynq-7000 / Ultra96) có sẵn RAM DDR ngoài, tích hợp module Wi-Fi/ESP32 để gửi cảnh báo đám mây (Cloud/IoT).

VII. CHÚ THÍCH

STT	Viết tắt	Tiếng Anh đầy đủ	Nghĩa tiếng Việt
1	AI	Artificial Intelligence	Trí tuệ nhân tạo
2	BRAM	Block RAM	Bộ nhớ RAM khối
3	CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
4	CMOS	Complementary Metal-Oxide-Semiconductor	Công nghệ bán dẫn CMOS
5	CPU	Central Processing Unit	Bộ xử lý trung tâm
6	DDR	Double Data Rate	Bộ nhớ DDR

7	DSP	Digital Signal Processing	Xử lý tín hiệu số
8	DVP	Digital Video Port	Giao tiếp dữ liệu video số
9	FC	Fully Connected	Kết nối đầy đủ
10	FPGA	Field-Programmable Gate Array	Mảng cổng lập trình được
11	FPS	Frames Per Second	Số khung hình trên giây
12	GPU	Graphics Processing Unit	Bộ xử lý đồ họa
13	HD	High Definition	Độ phân giải cao
14	HDL	Hardware Description Language	Ngôn ngữ mô tả phần cứng
15	HREF	Horizontal Reference	Tín hiệu tham chiếu ngang
16	IEEE	Institute of Electrical and Electronics Engineers	Viện Kỹ sư Điện và Điện tử
17	IoT	Internet of Things	Internet vạn vật
18	IP	Intellectual Property	Lõi thiết kế phần cứng
19	LED	Light-Emitting Diode	Điốt phát quang
20	MAC	Multiply-Accumulate	Phép nhân–tích lũy
21	PCLK	Pixel Clock	Xung đồng hồ pixel
22	RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
23	ROM	Read-Only Memory	Bộ nhớ chỉ đọc
24	ROI	Region of Interest	Vùng quan tâm
25	RTL	Register-Transfer Level	Mức truyền thanh ghi
26	SCCB	Serial Camera Control Bus	Bus điều khiển camera nối tiếp
27	SoC	System on Chip	Hệ thống trên chip
28	VGA	Video Graphics Array	Chuẩn giao tiếp hiển thị
29	VHDL	VHSIC Hardware Description Language	Ngôn ngữ mô tả phần cứng
30	VSYNC	Vertical Synchronization	Đồng bộ dọc

VIII. TÀI LIỆU THAM KHẢO

[1] Digilent, “Basys 3 AMD Artix-7 FPGA Trainer Board.” [Online]. Available: https://digilent.com/shop/basys-3-amd-artix-7-fpga-trainer-board-recommended-for-introductory-users/

[2] AMD, “7 Series FPGAs Data Sheet: Overview (DS180).” [Online]. Available: https://docs.amd.com/api/khub/documents/2LByHkO~nSZXcei2D55fTg/content?

[3] AMD, “7 Series FPGA CLB Resources (UG474).” [Online]. Available: https://docs.amd.com/r/en-US/ug474_7Series_CLB/7-Series-FPGA-CLB-Resources

[4] AMD, “7 Series DSP48E1 Slice User Guide (UG479).” [Online]. Available: https://docs.amd.com/api/khub/documents/gu4oRPFEh_Pm2uaAlfY6Kg/content?

[5] OmniVision – OV7675/OV7670 product information .” [Online]. Available: https://www.ovt.com/products/ov7675/?

[6] IEEE Xplore – Methodology for CNN Implementation in FPGA-Based Embedded Systems. [Online]. Available: https://ieeexplore.ieee.org/document/9810331

[7] IEEE Xplore – FPGA Implementation of Quantized Convolutional Neural Networks [Online]. Available: https://ieeexplore.ieee.org/document/8947168

[8] R. Krishnamoorthi, *Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper*, 2018. [Online]. Available: https://arxiv.org/abs/1806.08342?